

Installation for Automated Robots Go Swarming (ARGoS) Robot Simulator on Windows

Compiled by: Eric Rodriguez

This document is meant to assist with installing and setting up ARGoS3 robot simulator on a machine running Windows 10. The steps involved with this process include:

1. Installing Windows Subsystem for Linux and Linux environment
2. Installing required dependencies for Linux environment
3. Setting up a GUI
4. Installing ARGoS3
5. Running ARGoS3-examples (simulations)

1. Installing and setting up Linux on Windows 10:

Installing Windows Subsystem for Linux (WSL):

ARGoS is only compatible with Linux and Mac systems; however, by setting up a Linux environment on Windows, a user can run ARGoS. Running ARGoS3 on Windows requires a Linux environment, hence the first step is installing WSL. If you already have it installed, skip this section, and move to setting up the Linux environment.

Please note that this requires Windows 10.

First, we will need to install and activate the Windows Subsystem for Linux (WSL).

Open Windows Powershell as administrator.

Within the Windows Powershell terminal, type, or copy and paste:

dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

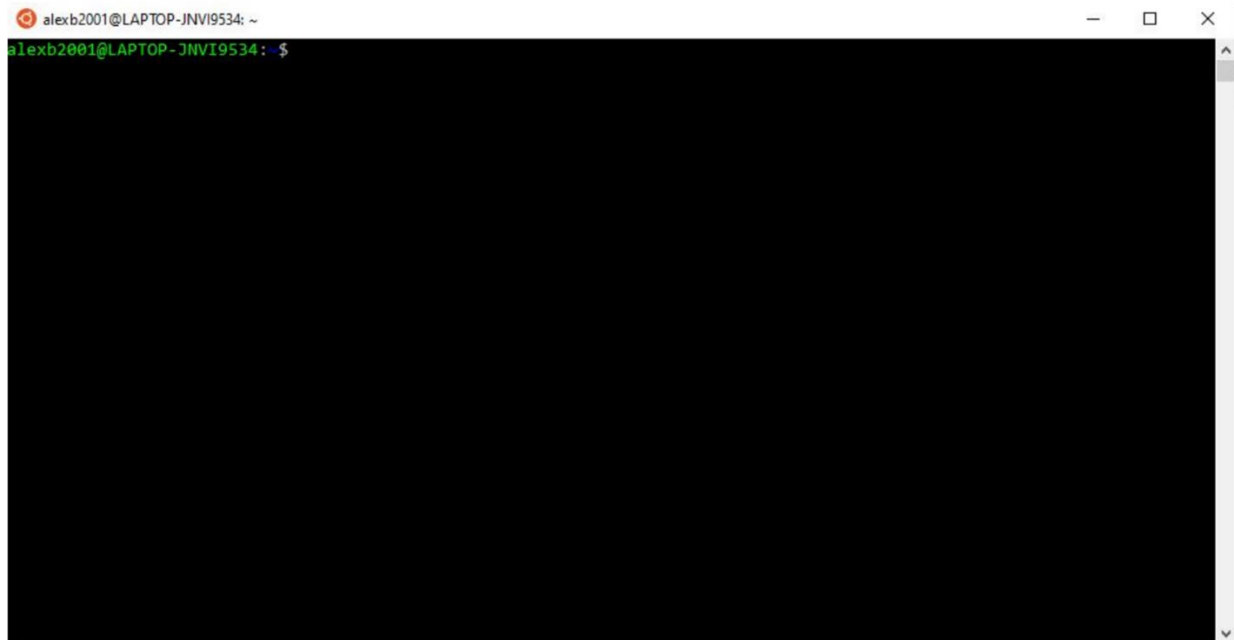
Run this command and then restart your computer.

To verify WSL is on Windows, open a command prompt and type **wsl -l -v**. This will display Linux environments running, or stopped, on your Windows machine, along with which version of WSL your Linux environment is using. If there are none, you will receive a message stating that there are no installed distributions.

Setting up the Linux environment (Ubuntu):

Now, install a Linux environment that will be used to run ARGoS. We will be using **Ubuntu 22.04**. Open the Microsoft Store app, and search Ubuntu 22.04 ((It will be updated to the latest version. Please check it on Ubuntu website).

Click install, and after installation is complete, click launch. The Ubuntu terminal will show up, which will prompt you to create a username and password. Once you have done this, your terminal should look like this:



Here is a link for additional information on using Ubuntu and other Linux environments with WSL:
<https://adamtheautomator.com/windows-subsystem-for-linux/>

Setting your WSL Version:

You might have to enable Ubuntu to work with WSL version 2. If so, you can do so by following the below instructions. If a user runs a Linux environment in WSL version 1, they may not be able to ping external networks, including Ubuntu archives and google.com.

Change Ubuntu to work on WSL version 2:

In a PowerShell console, list which versions of Linux you have installed by running **wsl -l** or **wsl --list**. Once you have the list, note the name of the Linux distro you want to run with WSL2.

Then run: **wsl --set-version <Distro> 2**

Replace <Distro> with the Linux distro. For this setup, <Distro>=**Ubuntu-22.04** ((It will be updated to the latest version. Please check it on Ubuntu website))

Confirm the command was successful by running **wsl -l -v** or **wsl --list --verbose** and noting that the Linux environment is now running on WSL version 2.

Verify that Ubuntu can run **sudo apt-get update** and **sudo apt-get install** by pinging google.com or us.archive.ubuntu.com. Then try running both commands if you are able to ping google or ubuntu.com.

2. Installing required dependencies

Once WSL is up and running, you can proceed by installing the required repositories.

Using Ubuntu environment, navigate to directory: **/etc/apt/** and view **sources.list** by **cat sources.list** and verify that it is not empty.

Next, you'll need to update your **sources.list** (a set of repositories your Linux system calls from). The list of repositories you will need are:

- **deb http://us.archive.ubuntu.com/ubuntu/ trusty main restricted universe**
- **deb-src http://us.archive.ubuntu.com/ubuntu/ trusty main**
- **deb http://us.archive.ubuntu.com/ubuntu/ trusty-security main restricted universe**
- **deb http://us.archive.ubuntu.com/ubuntu/ trusty-updates main restricted universe**
- **deb http://us.archive.ubuntu.com/ubuntu/ trusty-backports main restricted universe**

To update **sources.list** with these depositories, go to your Ubuntu terminal and type:

sudo add-apt-repository '[repo name]'

In place of '[repo name]', paste the full link of each repository (**make sure to keep the single quotes**).

For the deb-src repository however, you will not be able to use the "sudo add-apt-repository" to install it.

Instead, in your Ubuntu terminal type: **sudo vim /etc/apt/sources.list**

You will be greeted with the list of already added repositories. Scroll to the bottom of this list, then hit "i" to enter INSERT mode. Paste in the link on a new line and put a comment above it. Use # to comment a label for it. When done, it should look like:

#deb-src http://us.archive.ubuntu.com/ubuntu/ trusty main

deb-src http://us.archive.ubuntu.com/ubuntu/ trusty main

Then hit the escape key to exit INSERT mode, then enter Shift+ZZ or :qw to write and save to sources.list and exit back to the terminal. The following link shares tips for writing to and saving files using Vim text editor in Ubuntu: <https://stackoverflow.com/questions/11828270/how-do-i-exit-vim>

Start by running **sudo apt-get upgrade** and **sudo apt-get update** which will update all existing repositories. You may need to enter your password. You may also need to manually accept each update, which can be done by pressing the 'y' on your keyboard and hitting Enter.

3. Setting up a GUI:

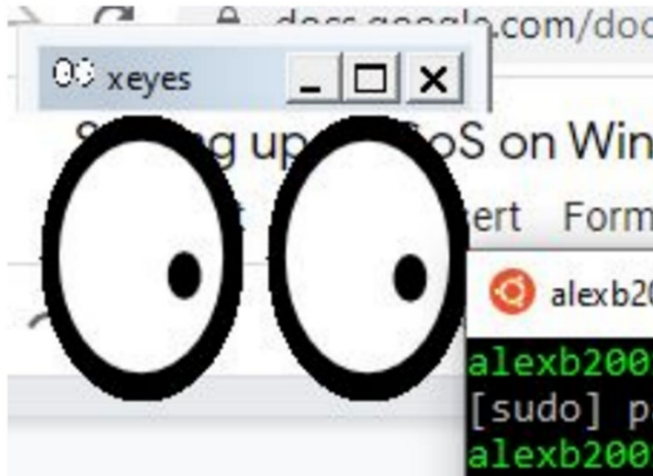
Run the following commands to establish a connection between WSL and the Windows display.

```
echo "export DISPLAY=:0" >> ~/.bashrc
source ~/.bashrc
```

Now, in Windows, install **Xming** from: <https://sourceforge.net/projects/xming/>
Launch Xming, and in your Linux terminal, run **xeyes** to ensure it is working.
Upon launching Xming, an icon should appear in your taskbar. This means Xming is running. To close Xming, simply right-click on this icon and hit "exit."



Once **xeyes** is executed, a new window should appear with a pair of eyes that will follow your mouse cursor. If this step is successful, move on to the next section. If not, make sure you typed the commands correctly and read the instructions below.



Errors with running **xeyes**:

If **xeyes** is not be able to run on Ubuntu, it may ask to download **x11-apps** instead. Go ahead and install x11-apps, which is equivalent to xeyes and several other X-server applications, by running:
sudo apt install x11-apps

There may a possibility to install and run **VcXsrv** at this link: <https://sourceforge.net/projects/vcxsrv/>
VcXsrv is very similar to Xming/XLaunch and works well with WSL+Ubuntu combination. Be sure to set a connection between the used IP for XLaunch and what Ubuntu has set for your route ip. You can view what IPs Ubuntu is using by executing: **ifconfig** or **route**. The ifconfig command is used to assign an address to a network interface and the route command is used manipulating the kernel's IP routing tables. You can also view etc/resolv.conf which contains hostnames and IP addresses for the system's Domain Name Server (DNS).

Use the below command to set either the IPs from `/etc/resolv.conf` or from your **route-ip**, respectively. StackOverflow has a discussion thread related to this issue that may assist with troubleshooting the “DISPLAY” parameter for running Graphical Linux Desktop Applications in Windows 10:

<https://stackoverflow.com/questions/61860208/wsl-2-run-graphical-linux-desktop-applications-from-windows-10-bash-shell-error>

Try these commands to establish a connection between WSL and Ubuntu:

```
export DISPLAY=$(grep nameserver /etc/resolv.conf | awk '{print $2}'):0.0
```

```
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0
```

```
export DISPLAY=$(route.exe print | grep 0.0.0.0 | head -1 | awk '{print $4}'):0.0
```

You will also need to **include inbound rules for VcXsrv** within your Windows Defender Firewall. Follow the path to include rules for VcXsrv:

Control Panel > System and Security > Windows Defender Firewall > Advanced Settings > Inbound Rules > New Rule... > Program > %ProgramFiles%\VcXsrv\vcxsrv.exe > Allow the connection > checked Domain/Private/Public > Named and Confirmed Rule.

StackExchange has a decent solution to correct “Can’t open display: 0” error message and if xeyes fails to execute correctly.

<https://superuser.com/questions/1476086/error-cant-open-display-0>

Mainly, you want to use the IP destination within route-ip, which may also be similar to the IP/nameserver identified in `/etc/resolv.conf`.

If issues persist, be sure to **enable X11 forwarding** within `/etc/ssh/sshd_config`. Edit this file in text editor and uncomment/remove # at line where X11Forwarding (yes/no) and be sure to change it to yes.

Skoshy has another solution to ensuring XLaunch is running properly on WSL2+Ubuntu at this link:

<https://github.com/microsoft/WSL/issues/4106>

4. Installing ARGoS3

Now that Linux has been installed successfully, we can begin on setting up ARGoS. It is recommended that you install ARGoS in the home folder within your Linux environment. Use `cd ~` to navigate to your home folder and use `mkdir src` to create the folder we’ll be working in. Navigate to it using `cd src`. In

your Ubuntu terminal, enter the following: **git clone** <https://github.com/ilpincy/argos3.git> **argos3** to make a local copy of ARGoS3.

Next, you'll need to install the requirements for compiling, installing and running ARGoS. To collect requirements simply enter: **sudo apt-get install <requirement>**. Substitute <requirement> with the following libraries:

```
cmake  
libfreeimage-dev  
libfreeimageplus-dev  
qt5-default  
freeglut3-dev  
libxi-dev  
libxmu-dev  
liblua5.3-dev  
lua5.3  
doxygen  
graphviz  
libgraphviz-dev  
asciidoc
```

We can now begin to compile ARGoS. You should now have an “**argos3**” folder (to check, type “ls”). To compile ARGoS, run the following commands:

```
$ cd argos3  
$ mkdir build_simulator  
$ cd build_simulator  
$ cmake ../src  
$ make
```

Finally, we can install ARGoS. At this point, you should be within “~/argos3/build_simulator”.

While staying in this directory, enter the following:

```
$ make doc  
$ sudo make install
```

(Note: the compilation, **make doc** and **make install** steps will take some time to complete, so just be patient.)

Note that you should **remain in the build_simulator** directory.

Once this is all complete, ARGoS should be installed on your system. Now, all we have left to do is download and test your examples to make sure everything is working correctly.

5. Running ARGoS3-examples (simulations)

To run ARGoS examples, you'll want to make sure you're in the “**build_simulator**” directory, and enter the following: **git clone** <https://github.com/ilpincy/argos3-examples.git>. This will create a local copy of example files within the build_simulator directory.

Use **cd argos3-examples** to navigate to the newly created directory.

Compile the examples by running the following commands:

```
mkdir build
```

```
cd build
```

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

After the “**cmake**” line, you may encounter an error saying you are missing “google perf-tools.”

To fix this, enter the following commands:

```
sudo apt-get install google-perftools
```

```
sudo apt-get install libgoogle-perftools4
```

Once these packages are installed, try cmake again. **If successful, type “make”,** and wait for the compilation process to complete. If during the “make” operation you encounter an error that says “**argos3_simulator could not be found**”, type:

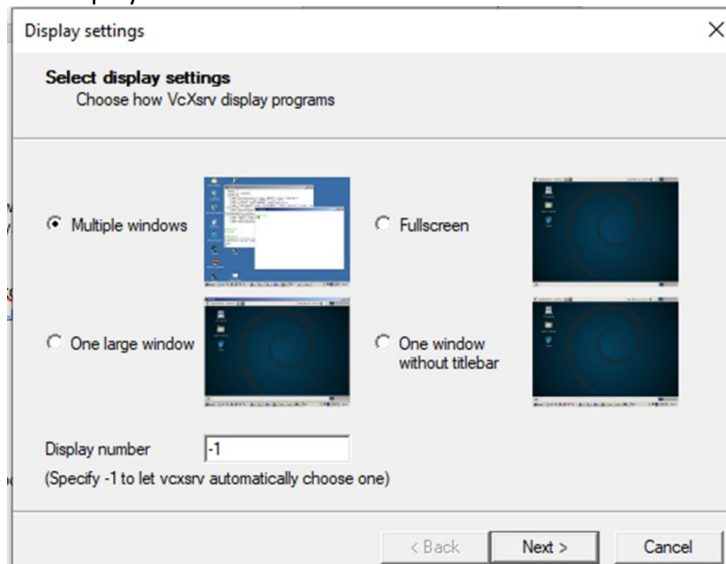
```
export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig/
```

And attempt the process again.

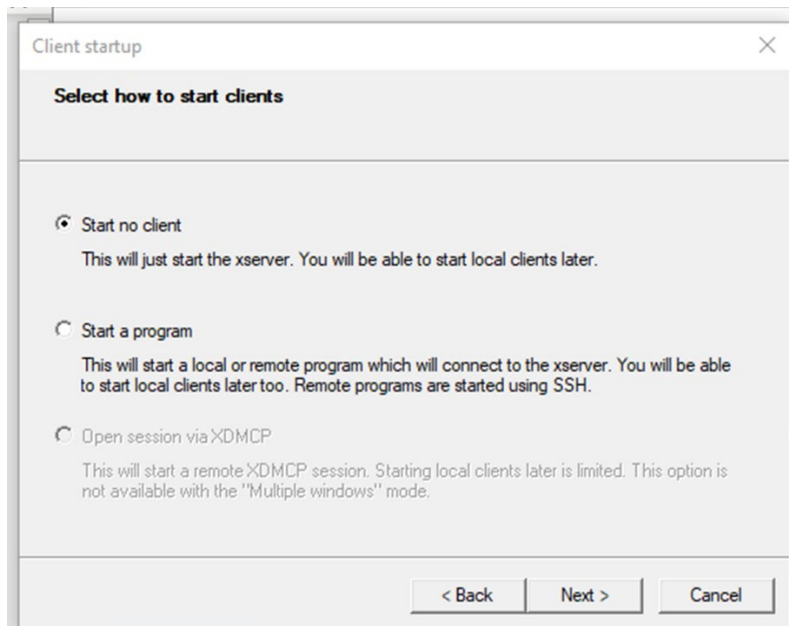
After executing “make” in the “build” directory, **step back one folder in the directory by typing “cd ..”**.

Argos3-examples have to be executed outside of the “build” directory and in the same directory containing CMakeLists.txt, which contains instructions for compiling and executing ARGoS.

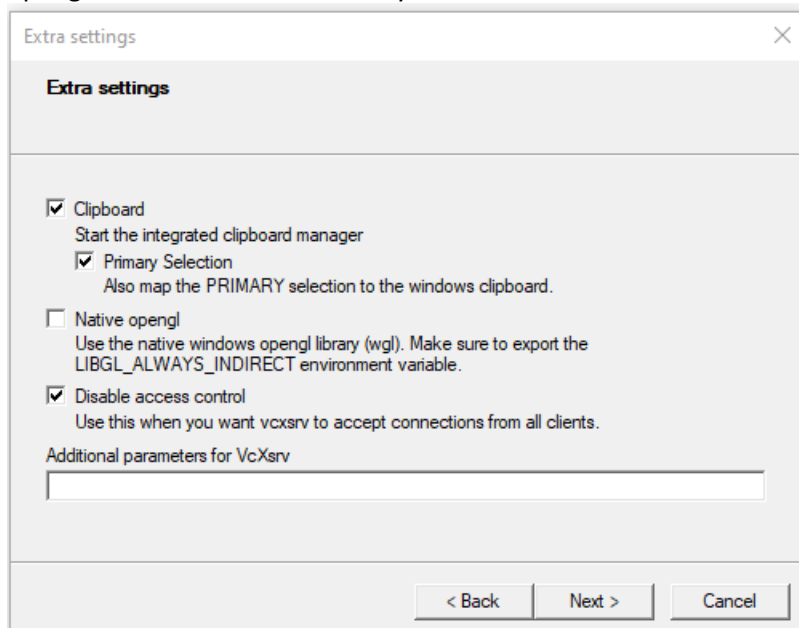
Launch XLaunch and choose one window in the display settings. If you are running through VcXsrv you’ll see Display number as -1. Then hit next.



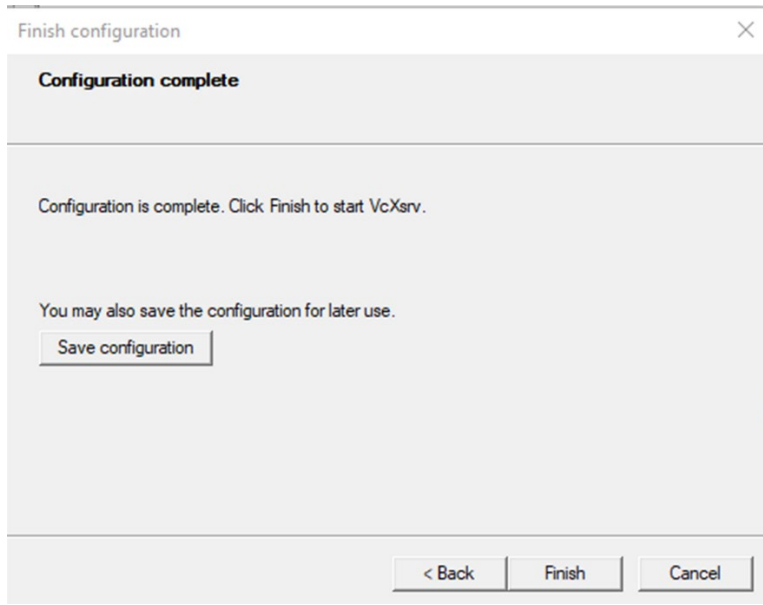
Next, choose Start no client for Client Startup.



Within “Extra settings” be sure to select “Clipboard” along with “Primary Selection”. Ensure “Native opengl” is not selected and finally ensure “Disable access control” is selected.

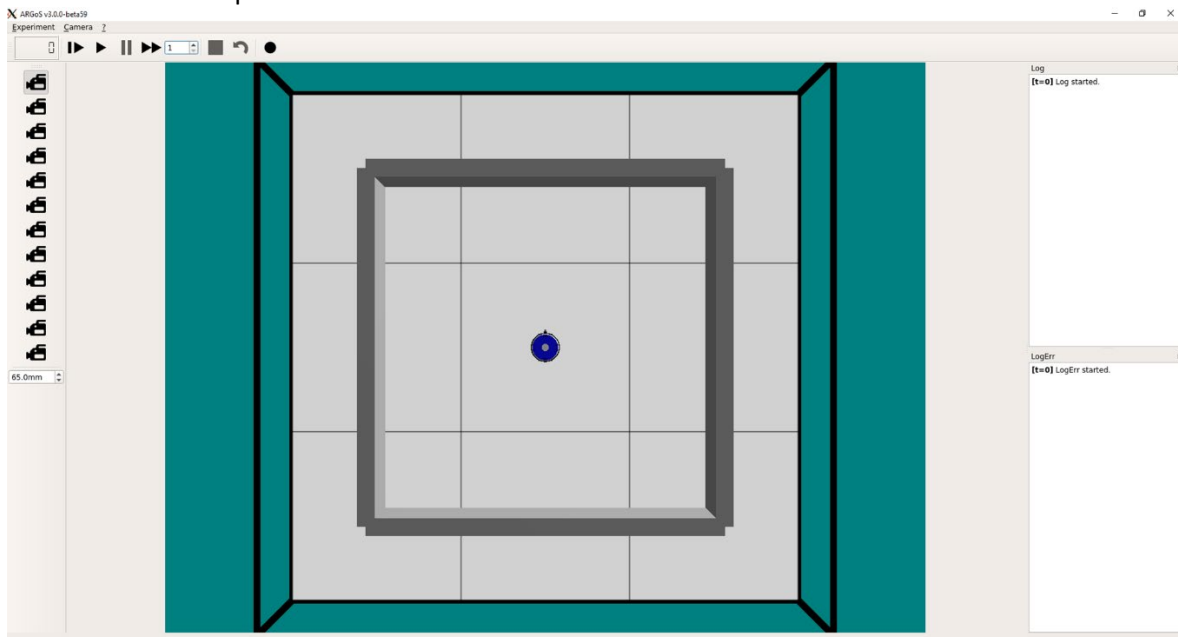


Lastly, be sure to save your configuration file for future setup.

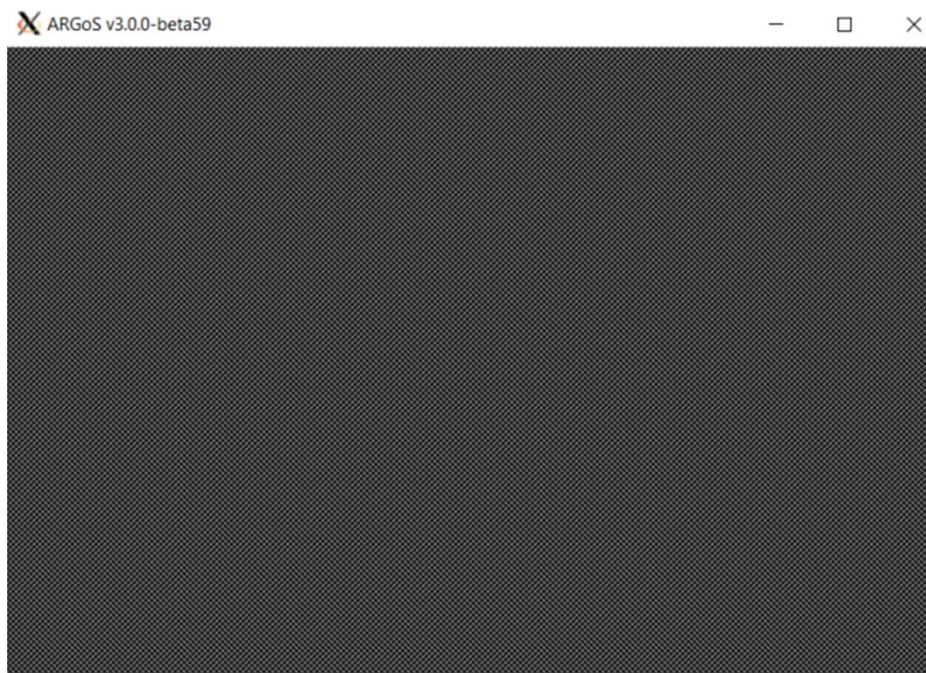


To simulate an experiment, run `argos3 -c 'experiment-name'` (replace 'experiment-name' with the file you are trying to simulate).

A window should open that looks like this:



If instead it looks like this:



Try restarting your computer, launching Xming, and then reopening WSL and trying again. The software is somewhat finnickily.

To run an ARGoS example, make sure Xming is on, and enter one of the following commands:

```
$ argos3 -c experiments/diffusion_1.argos  
for the example obstacle avoidance with a single robot,
```

```
$ argos3 -c experiments/diffusion_10.argos  
for the example obstacle avoidance with 10 robots,
```

```
$ argos3 -c experiments/synchronization.argos  
for the synchronization experiment,
```

```
$ argos3 -c experiments/flocking.argos  
for the flocking experiment, and
```

```
$ argos3 -c experiments/foraging.argos
```